

Un LLM dans l'algorithme A* sans forcément casser les garanties théoriques ?

ABDELOUHAB Yacine et JANIN Paul
M2 IASD App
Projet Modélisation de Problèmes

Juillet 2026

Résumé

A* recherche un chemin entre un état de départ et un état but dans un graphe d'états. Il garantit de trouver le chemin de coût minimal parmi tous les chemins possibles entre ces deux états. Pour cela, A* évalue chaque état par une fonction f , qui additionne le coût déjà payé pour atteindre l'état et une estimation du coût restant appelée heuristique. Trois points d'insertion se distinguent naturellement dans l'algorithme. On se demande alors s'il est possible d'y insérer un LLM sans casser la garantie d'optimalité, et si oui à quel prix. Deux de ces trois points peuvent être rendus théoriquement sûrs moyennant des protections précises. Le gain mesuré reste pourtant nul ou négligeable, même sur les niveaux faciles.

On casse ensuite volontairement ces garanties en reproduisant le mécanisme du papier LLM-A*, où le LLM n'est plus protégé du tout. Le gain en efficacité est net sur la recherche de chemin, mais nettement plus incertain sur Taquin et Sokoban, où l'espace de dispositions possibles est bien plus grand. Sur l'instance la plus difficile de Taquin, cette liberté totale casse même l'optimalité et ralentit la recherche au lieu de l'accélérer.

Table des matières

1	Introduction	3
1.1	Résumé de A*	3
1.2	Identification des points d'entrée pour un LLM	4
1.3	Le mécanisme du papier LLM-A*	4
1.4	Question de recherche	4
1.5	Plan du rapport	4
2	Un algorithme A* générique et trois points d'insertion	5
2.1	Quel état choisir pour l'exploration ? (point d'insertion 1)	5
2.2	Peut on remplacer l'heuristique par un LLM ? (point d'insertion 2)	5
2.3	Peut on laisser un LLM écarter des états ? (point d'insertion 3)	8
3	Reproduction du mécanisme du papier LLM-A*	9
3.1	Définitions d'un pathway pour chacun des jeux	9
3.2	Un coût en appels API très différent	9
4	Expérimentation	9
4.1	Définitions des heuristiques	9
4.2	Protocole	10
4.3	Résultats pour Recherche de chemin	11
4.4	Résultats pour Taquin	12
4.5	Résultats pour Sokoban	13
4.6	Réponse à la problématique	13

1 Introduction

1.1 Résumé de A*

A* est un algorithme de recherche qui trouve le chemin le moins coûteux entre un état de départ et un état but, dans un ensemble d'états reliés entre eux par des actions possibles. Pour chaque état qu'il découvre, A* retient un coût g , qui est le prix déjà payé pour l'atteindre depuis le départ, et une estimation h du coût qui reste à payer pour atteindre le but depuis cet état.

Cette estimation h est appelée **l'heuristique**, et son rôle est de guider la recherche vers le but sans avoir à tout explorer. A* combine ces deux quantités en une valeur $f = g + h$ pour chaque état, et explore toujours en priorité l'état de plus petite valeur de f parmi ceux qu'il connaît déjà mais n'a pas encore explorés.

Cette règle simple donne à A* sa **garantie d'optimalité**. Si l'heuristique ne surestime jamais le vrai coût restant, alors le premier état but que A* explore est nécessairement atteint par le chemin le moins coûteux possible. La figure 1 illustre ce mécanisme sur un petit exemple.

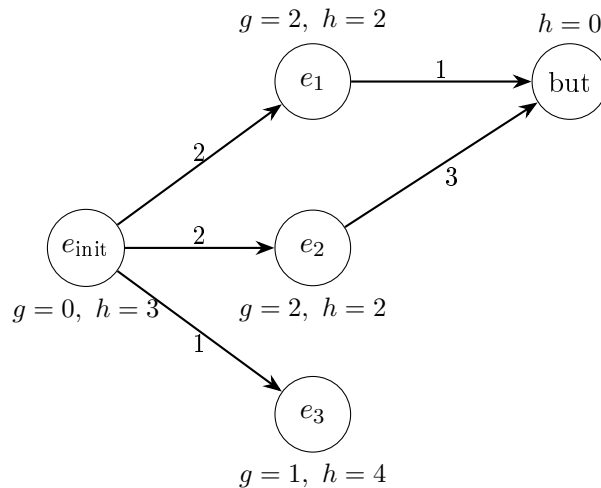


FIGURE 1 – Exemple jouet, en deux temps.

Premier temps

Depuis e_{init} , A* génère trois voisins.

- e_1 : coût 2, reste estimé 2, $f(e_1) = 4$.
- e_2 : coût 2, reste estimé 2, $f(e_2) = 4$, égalité avec e_1 .
- e_3 : coût 1, reste estimé 4, $f(e_3) = 5$.

e_1 et e_2 sont à égalité, avec le f le plus petit. A* choisit arbitrairement l'un des deux, par exemple e_1 , et laisse e_2 disponible.

Deuxième temps

A* explore e_1 et découvre *but*, à coût 1 de plus, $f(\text{but}) = g(e_1) + 1 + 0 = 3$. C'est désormais la plus petite valeur parmi les états à explorer (e_2 à $f = 4$, e_3 à $f = 5$), donc A* explore *but* et s'arrête.

1.2 Identification des points d'entrée pour un LLM

L'algorithme A^* prend, à chaque étape, plusieurs petites décisions, et certaines d'entre elles pourraient en principe être confiées à un grand modèle de langage plutôt qu'à une règle fixe. Trois de ces décisions se distinguent naturellement.

- Le choix de l'état à explorer en premier.
- Le calcul de l'estimation h elle-même, qui pourrait être produite par un LLM à partir d'une description de l'état, plutôt que calculée par une formule fixe.
- La décision d'écarter définitivement un état jugé sans issue, avant même de l'explorer.

Ces trois décisions n'ont pas le même effet sur la garantie d'optimalité de A^* , et c'est précisément ce que ce rapport étudie en détail dans la section 2.

1.3 Le mécanisme du papier LLM- A^*

Le papier **LLM- A^*** [1] propose une approche différente, qui n'agit sur aucune des trois décisions ci-dessus. Il fait générer par un LLM une liste d'états appelés **waypoints**. Il s'agit de points de passage intermédiaires plausibles entre le départ et le but. On modifie ensuite directement la fonction d'évaluation de A^* en y ajoutant un troisième terme

$$f(s) = g(s) + h(s) + \text{cout_vers_cible}(\text{cible actuelle}, s)$$

où `cout_vers_cible` mesure la distance restante entre l'état s et le prochain waypoint à atteindre, et où le waypoint courant change au fil de la recherche à mesure que les précédents sont atteints.

Ce troisième terme guide donc activement la recherche vers le prochain waypoint plutôt que directement vers le but. Il est cependant fourni par un LLM, sans aucune propriété garantie, ni admissibilité ni consistance, et rien dans la méthode du papier ne borne l'écart entre la solution trouvée et l'optimum réel.

1.4 Question de recherche

Ce projet part d'une idée inverse. Peut-on insérer un LLM à des points précis d'un algorithme A^* générique, choisis parce qu'on peut analyser leur impact sur l'optimalité, et parfois le neutraliser ? Et si oui pour certains points mais pas pour d'autres, quel est le prix réel de cette sûreté, en appels API et en gain mesuré ?

Nous répondons à cette question sur trois jeux, la recherche de chemin sur grille, le taquin, et Sokoban. Ces trois jeux diffèrent par le nombre d'actions possibles à chaque état, par la fréquence à laquelle un même état est atteignable par plusieurs chemins différents, et par la présence ou non de véritables impasses.

1.5 Plan du rapport

La section 2 détaille les trois points d'insertion identifiés, avec pour chacun une analyse théorique de sa garantie. La section 3 reproduit et étend le mécanisme du papier LLM A^* sur nos trois jeux. La section 4 présente les résultats de nos expérimentations, et y répond directement.

2 Un algorithme A* générique et trois points d'insertion

L'algorithme A* du projet (`Algorithmes/A_etoile/a_etoile.py`) est écrit pour rester générique, et permet de faire intervenir un LLM aux trois endroits identifiés dans l'introduction.

1. **Ordonner les nœuds à explorer** (`departager_llm`), où le LLM choisit, parmi des états à *f strictement égal*, lequel explorer en premier.
2. **Calculer l'heuristique d'un nœud** (`heuristique_llm_lot`), où le LLM remplace entièrement $h(\text{état})$.
3. **Élaguer les états** (`elaguer_llm`), où le LLM décide si un état candidat doit être définitivement écarté des ouverts.

Ces trois points n'ont pas le même statut vis-à-vis de la garantie d'optimalité, et c'est précisément ce qui en fait un objet d'étude intéressant.

2.1 Quel état choisir pour l'exploration ? (point d'insertion 1)

A* doit décider, à chaque étape, lequel des états déjà connus mais pas encore explorés il va traiter ensuite¹. Un LLM pourrait sembler bien placé pour cette décision, à partir d'une description de chaque candidat. Le problème est que rien ne garantit la façon dont il va trancher parmi cette liste. L'optimalité de A* n'est garantie que si l'état retenu est toujours celui de plus petite valeur de f .

Or, en pratique, plusieurs états partagent souvent cette plus petite valeur de f , et pas un seul. Ceci est visible sur l'exemple de la figure 1, mais aussi dans les jeux considérés ici. Si on force donc le LLM à ne recevoir en entrée que cet ensemble d'états à f minimal, et à trancher uniquement parmi eux, l'optimalité reste garantie, par construction.

C'est exactement ce que fait `departager_llm`. On modélise l'ensemble des états à explorer par un tas (`heapq`), on dépile ceux qui partagent la même valeur de f , et on les propose au LLM, qui choisit lequel explorer en premier. Il ne choisit donc que parmi les états ayant le score le plus petit. Les états non choisis sont remis dans le tas. En pratique, ce gain reste toutefois modeste, et même nul sur les niveaux les plus difficiles, un point développé en section 4.

2.2 Peut on remplacer l'heuristique par un LLM ? (point d'insertion 2)

A* a besoin d'une estimation du coût restant pour chaque état, c'est l'heuristique h . Un LLM pourrait sembler bien placé pour trouver cette estimation, à partir d'une description de l'état, plutôt que de la calculer avec une formule fixe. C'est exactement ce que fait `heuristique_llm_lot`, qui remplace entièrement $h(\text{état})$. On présente au LLM l'ensemble des états voisins nouvellement découverts, au moment de l'expansion d'un nœud, et on lui demande d'estimer le coût restant pour chacun.

Contrairement au point 1, ce choix change directement quelle valeur de f est la plus petite. La garantie d'optimalité n'est donc plus automatique. Elle dépend de propriétés que le LLM n'a aucune raison de respecter spontanément.

1. Dans notre code, cet ensemble correspond à la liste `ouverts`, et les états déjà traités, dans `fermes`, ne sont plus reconsidérés.

Deux propriétés indépendantes

Soit $h^*(\text{état})$ le vrai coût minimal restant depuis un état jusqu'à un but.

Une heuristique h est **admissible** si, pour tout état, $h(\text{état}) \leq h^*(\text{état})$, c'est à dire si elle ne surestime jamais.

Elle est **consistante**, une propriété plus forte, si pour tout état et tout voisin accessible en un pas de coût $c(\text{état}, \text{voisin})$, $h(\text{état}) \leq c(\text{état}, \text{voisin}) + h(\text{voisin})$.

La consistance implique l'admissibilité, mais l'inverse est faux.

Cette distinction est cruciale pour notre code parce que notre `a_etoile()` ne rouvre jamais un état une fois placé dans `fermes` (`if etat_suivant in fermes: continue`), une simplification standard qui n'est correcte que sous l'hypothèse de **consistance**. Avec une heuristique seulement admissible mais pas consistante, un état peut être fermé avec un g non optimal, et un chemin moins cher découvert plus tard est alors purement ignoré. La consistance, et elle seule, garantit qu'un état fermé a toujours son coût optimal.

Contre-exemple

La figure 2 montre le graphe construit pour ce test, avec l'heuristique h indiquée sous chaque état.

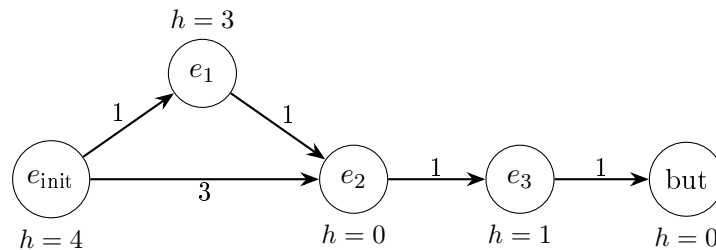


FIGURE 2 – Illustration de l'importance de la consistance

Le vrai optimum est $e_{\text{init}} \rightarrow e_1 \rightarrow e_2 \rightarrow e_3 \rightarrow \text{but}$, de coût 4. Vérifions l'admissibilité, état par état, avec $h^*(\text{état})$ le vrai coût restant jusqu'à *but*.

- $h(e_{\text{init}}) = 4 \leq h^*(e_{\text{init}}) = 4$.
- $h(e_1) = 3 \leq h^*(e_1) = 3$.
- $h(e_2) = 0 \leq h^*(e_2) = 2$.
- $h(e_3) = 1 \leq h^*(e_3) = 1$.
- $h(\text{but}) = 0 \leq h^*(\text{but}) = 0$.

Cette heuristique est donc admissible partout. Elle n'est cependant pas consistante. Prenons l'arête $e_1 \rightarrow e_2$. La consistance exigerait $h(e_1) \leq c(e_1, e_2) + h(e_2)$, soit $3 \leq 1 + 0 = 1$, ce qui est faux. Suivons maintenant, pas à pas, ce que fait notre `a_etoile()` sur ce graphe.

- **Premier temps**, depuis e_{init} , $f(e_1) = 1 + 3 = 4$ et $f(e_2) = 3 + 0 = 3$. A* explore e_2 en premier (f plus petit), et le ferme avec $g(e_2) = 3$. On explore donc plus jamais e_2 !
- **Deuxième temps**, A* explore e_1 et le ferme avec $g(e_1) = 1$. Il redécouvre e_2 par un chemin moins cher ($g = 2$ au lieu de 3), mais e_2 est déjà fermé, donc l'amélioration est ignorée (`if etat_suivant in fermes: continue`).
- **Troisième temps**, A* continue avec le $g(e_2) = 3$ erroné, explore e_3 puis *but*, et s'arrête sur un coût de 5, au lieu du vrai optimum de 4.

Une heuristique LLM seulement admissible reste donc exposée à un second risque, plus sournois, à savoir que rien ne garantit sa consistance d'un état à son voisin.

Deux parades testées simultanément

Notons h_{llm} l'heuristique renvoyée par le LLM, et $h_{\text{classique}}$ l'heuristique classique du jeu, celle que `a_etoile()` utiliserait par défaut sans LLM. Chacune des deux parades ci-dessous protège une seule des deux propriétés.

Le $\min(h_{\text{llm}}, h_{\text{classique}})$ protège l'admissibilité par construction. L'heuristique classique $h_{\text{classique}}$ est déjà admissible, donc

$$h_{\text{classique}}(\text{état}) \leq h^*(\text{état}) \quad \text{pour tout état.}$$

Comme le minimum des deux est toujours inférieur ou égal à $h_{\text{classique}}$, on obtient directement

$$\min(h_{\text{llm}}, h_{\text{classique}})(\text{état}) \leq h^*(\text{état})$$

pour tout état, quelle que soit la valeur renvoyée par le LLM. Ce mécanisme ne protège cependant pas la consistance.

La consistance demande :

$$\text{pour chaque état, et chacun de ses voisins, } h(\text{voisin}) \geq h(\text{état}) - \text{coût}(\text{état}, \text{voisin}).$$

Le **pathmax** impose cette inégalité, mais seulement pour un seul voisin à la fois, celui par lequel A^* vient justement d'arriver, le parent. À chaque pas parent vers voisin,

$$h(\text{voisin}) = \max(h_{\text{llm}}(\text{voisin}), h(\text{parent}) - \text{coût}(\text{parent}, \text{voisin})).$$

Comme $h(\text{parent})$ est lui-même construit par la même règle depuis son propre parent, cette propriété se propage par récurrence à tout le chemin exploré depuis le départ. Le $\max()$ ne peut cependant jamais réduire une valeur. Pathmax ne corrige donc jamais une surestimation déjà présente dans h_{llm} .

Imposer une vraie consistance partout garantirait aussi l'admissibilité. Cela demanderait de connaître h pour tous les voisins d'un état, y compris ceux pas encore découverts par la recherche. Une recherche incrémentale ne peut pas anticiper ce calcul. La seule heuristique à la fois consistante et exacte serait h^* lui-même, ce qui reviendrait à résoudre le problème avant de le chercher. Pathmax se limite donc à la relation parent-voisin, la seule information disponible sans coût supplémentaire, jamais à tous les couples état-voisin. Ce n'est donc jamais une vraie consistance globale. Le minimum, de son côté, protège seulement l'admissibilité.

Puisque la consistance impliquerait l'admissibilité, pathmax ne la garantit pas non plus. Aucune des deux parades ne suffit donc seule. Nos expérimentations activent donc toujours les deux ensemble par défaut, pour que les coûts mesurés restent fiables.

Version par lot

On garde uniquement la version par lot, `heuristique_llm_lot`, pour toute la suite de ce rapport. Elle note tous les voisins nouveaux d'un même nœud en un seul appel API, au lieu d'un appel par état. Sans ça, le nombre d'appels API exploserait sur les niveaux réalistes.

2.3 Peut on laisser un LLM écarter des états ? (point d’insertion 3)

Contrairement aux points 1 et 2, `elaguer_llm` n’a aucune protection possible. Dans `a_etoile()`, `elaguer_llm` est appelé au moment où un nouvel état voisin est découvert, avant même qu’il entre dans les ouverts. On lui soumet cet état, sans aucune garantie que son jugement soit correct.

Si le LLM répond à tort que l’état est impossible, il est perdu pour toujours. Contrairement à une heuristique, qui influence seulement l’ordre d’exploration, une mauvaise décision ici peut supprimer la seule solution existante. A^* ne la retrouvera jamais.

Vérification empirique sur quelques exemples

On teste d’abord ce jugement sur des cas classiques, déjà bien identifiés, comme :

- A. Un coin mort certain, avec deux murs perpendiculaires et pas de cible. Il est bien impossible.
- B. Une caisse déjà sur sa cible. Il n’est pas impossible.
- C. Un bloc 2×2 de 4 caisses en pleine salle ouverte, un **freeze deadlock** où chaque caisse bloque les 3 autres sans qu’aucun mur ne soit nécessaire. Il est impossible, mais la règle classique ne le couvre pas, puisqu’aucun mur ne se trouve à proximité.
- D. Une caisse seule en pleine salle. Il n’est pas impossible.

Nous testons ces quatre états avec Claude Sonnet, selon deux variantes de prompt. Le **jugement libre** laisse le LLM juger librement. Le **raisonnement guidé** lui impose de vérifier explicitement chacune des 4 directions de poussée, pour chaque caisse.

Variante	A (coin)	B (cible)	C (freeze)	D (normal)
Jugement libre	Faux (raté)	Correct	Faux (raté)	Correct
Raisonnement guidé	Correct	Correct	Faux (raté)	Correct

TABLE 1 – Le jugement libre rate même le coin, le cas le plus évident. Le raisonnement guidé corrige ce cas. Il rate cependant encore le *freeze deadlock*.

Nous avons aussi testé et écarté une explication alternative, à savoir que le format de description de l’état changerait le résultat. Une grille détaillée avec des coordonnées explicites donne exactement le même résultat qu’une grille seule et minimale. Le raté n’est donc pas un problème de présentation du prompt, mais un vrai manque de vérification systématique de la mécanique de poussée.

Même sur ces cas classiques, pourtant simples, le LLM échoue déjà. C’est pourquoi la détection des cas classiques est placée clairement avant tout appel au LLM. Pour le problème du Sokoban, on repère ainsi une fois pour toutes ces cas simples et on laisse le LLM gérer les autres cas d’impossibilité.

L’élitage n’est testé que sur Sokoban, parmi nos trois jeux. Recherche de chemin et Taquin n’ont pas de vrai cul-de-sac. Tout état atteint y reste résoluble. Tester l’élitage dessus ne voudrait rien dire.

Au vu de ces résultats, élaguer avec un LLM va sûrement être compliqué, voire inefficace, surtout pour Sokoban, précisément le jeu où élaguer serait le plus utile, sur un espace d’états qui peut croître de façon exponentielle.

3 Reproduction du mécanisme du papier LLM-A*

Le papier LLM-A* laisse un LLM proposer librement des points de passage (waypoints), sans aucune garantie théorique, comme rappelé en section 2.2. Pour comparer notre approche à celle du papier original, nous avons reproduit et étendu ce mécanisme sur nos trois jeux. On veut savoir si, malgré la perte de la garantie d’optimalité, on gagne en efficacité en ne bridant pas le LLM comme le font les points d’insertion 1 et 2.

3.1 Définitions d’un pathway pour chacun des jeux

Un sous-objectif n’a de sens, pour Taquin ou pour Sokoban, qu’en tant qu’état complet. Nous généralisons donc le mécanisme du papier en **dispositions**, qui prennent une forme différente pour chacun de nos trois jeux.

Pour Recherche de chemin, une disposition est directement une coordonnée (x, y) , comme le waypoint du papier original.

Pour Taquin, une disposition est une permutation complète du plateau, puisqu’un sous-objectif partiel n’y a aucun sens.

Pour Sokoban, une disposition est un ensemble complet de positions de caisses, optionnellement augmenté de la position du joueur.

3.2 Un coût en appels API très différent

Ce mécanisme demande un seul appel au LLM par instance, fait une fois avant même de démarrer la recherche, pour obtenir la liste complète des dispositions. La recherche se déroule ensuite entièrement de façon classique, sans plus jamais consulter le LLM. C’est une différence de plusieurs ordres de grandeur avec les points d’insertion 1 et 2, qui consomment jusqu’à 150 appels par instance. Cet appel unique reste cependant sans aucune garantie, contrairement aux points 1 et 2, ce qui en fait un compromis inverse, peu coûteux mais risqué, plutôt que sûr mais coûteux.

4 Expérimentation

4.1 Définitions des heuristiques

Chaque jeu garde sa propre heuristique classique, admissible car construite comme un relâchement du vrai problème, une version plus simple qui ignore une vraie contrainte et ne peut donc que sous-estimer le coût restant. Pour Recherche de chemin, c’est la distance de Manhattan vers le but, qui ignore les murs. Pour Taquin, c’est la somme des distances de Manhattan de chaque pièce vers sa position finale, qui ignore qu’un coup ne déplace qu’une seule pièce à la fois. Pour Sokoban, c’est la somme des distances de Manhattan de la meilleure affectation caisse vers cible, calculée avec l’**algorithme hongrois** (`scipy.optimize.linear_sum_assignment`), qui ignore les murs et le fait qu’on ne pousse qu’une seule caisse à la fois.

4.2 Protocole

Le pipeline officiel (`Resultat/lancer_experimentation.py`) comprend 11 tâches indépendantes, une par instance, chacune comparant une **baseline** classique aux points d’insertion 1 et 2, ainsi qu’au point 3 pour Sokoban.

Sokoban est coupé en deux corpus distincts, microban et l’original de 1988. L’original est bien trop compliqué pour une recherche classique. Son espace d’états explose avant même de trouver une solution (table 2). Microban reste abordable, tout en gardant les mêmes vraies mécaniques du jeu.

Les points d’insertion 1 et 2 consultent le LLM à chaque décision, jusqu’à 150 fois par instance. Nous utilisons DeepSeek (`deepseek-v4-pro`) pour ce pipeline systématique. Claude aurait demandé le même nombre d’appels, à un coût bien plus élevé. Pour les waypoints, un seul appel suffit par instance. Nous testons donc les deux fournisseurs, Claude (`claude-sonnet-5`) et DeepSeek (`deepseek-v4-pro`).

Nous construisons nous-mêmes chaque instance à la main.

- Recherche de chemin, dans `Jeux/Recherche_de_chemin/exemples/`.
- Taquin, dans `Jeux/Taquin/exemples/`.
- Sokoban, dans `Jeux/Sokoban/exemples/`, pour microban et l’original.
- Le **plafond d’états** arrête la recherche, sans solution, si elle explore trop d’états, pour ne jamais attendre indéfiniment sur un espace qui explose. Il est de 100 000 pour les instances standard et de 250 000 pour Sokoban original.
- Le **budget d’appels API** est de 150 par variante. Une fois ce budget épuisé, chaque hook retombe sur son comportement classique pour le reste de la recherche, ce qui évite tout blocage et tout coût illimité.
- Les instances couvrent plusieurs niveaux pour chaque jeu, avec une ou plusieurs cartes différentes par niveau (table 2).

Jeu	niveau1	niveau2	niveau3	niveau4
Recherche de chemin	30–33	82–110	147–343	82–158
Taquin	4–17	34–83	18–68	12 061–14 606
Sokoban (microban)	1088–1088	1761–1761	13 428–13 428	56 240–56 240
Sokoban (original, 1988)	≥200 000, échec classique			

TABLE 2 – Nombre d’états explorés par A* classique, sans LLM, minimum–maximum sur les cartes testées par niveau. Sokoban microban n’a qu’une carte par niveau. Taquin niveau4 est exclu, trop coûteux.

4.3 Résultats pour Recherche de chemin

Instance	Variante	Coût	Visites	Appels API
niveau1_facile	A*	10	33	0
niveau1_facile	point 1 (departager_llm)	10	31	31
niveau1_facile	point 2 (heuristique_llm_lot)	10	33	33
niveau1_facile	LLM-A*	10	14 (Claude, DeepSeek)	1
niveau2_moyen	A*	21	110	0
niveau2_moyen	point 1 (departager_llm)	21	109	109
niveau2_moyen	point 2 (heuristique_llm_lot)	21	110	106
niveau2_moyen	LLM-A*	21	30 (Claude, DeepSeek)	1
niveau3_difficile	A*	42	171	0
niveau3_difficile	point 1 (departager_llm)	42	171	150 (budget épuisé)
niveau3_difficile	point 2 (heuristique_llm_lot)	42	171	150 (budget épuisé)
niveau3_difficile	LLM-A*	42	81 (Claude), 67 (DeepSeek)	1

TABLE 3 – Recherche de chemin, tous les mécanismes comparés. Points 1 et 2 avec DeepSeek. LLM-A* avec Claude et DeepSeek.

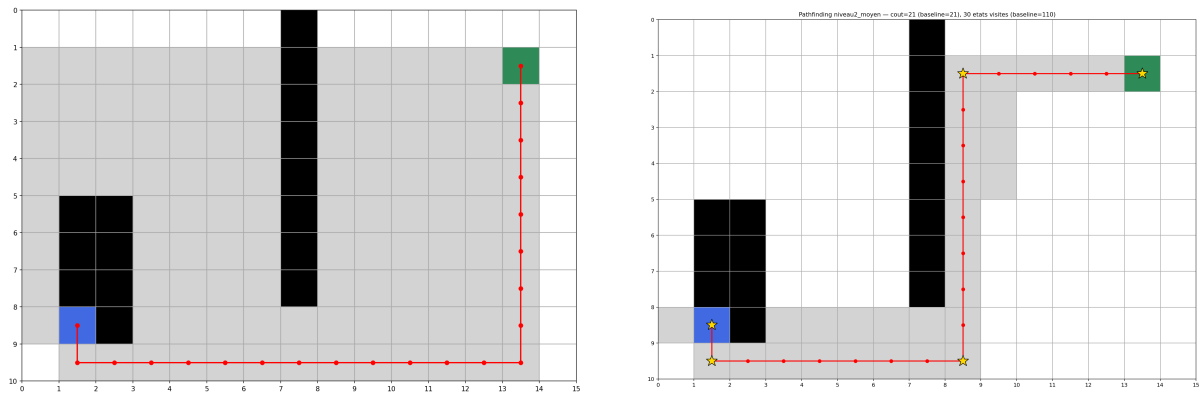


FIGURE 3 – Recherche de chemin niveau2_moyen. À gauche, A* (110 états visités). À droite, LLM-A* (30 états visités), à coût identique, avec de points de passage intermédiaires.

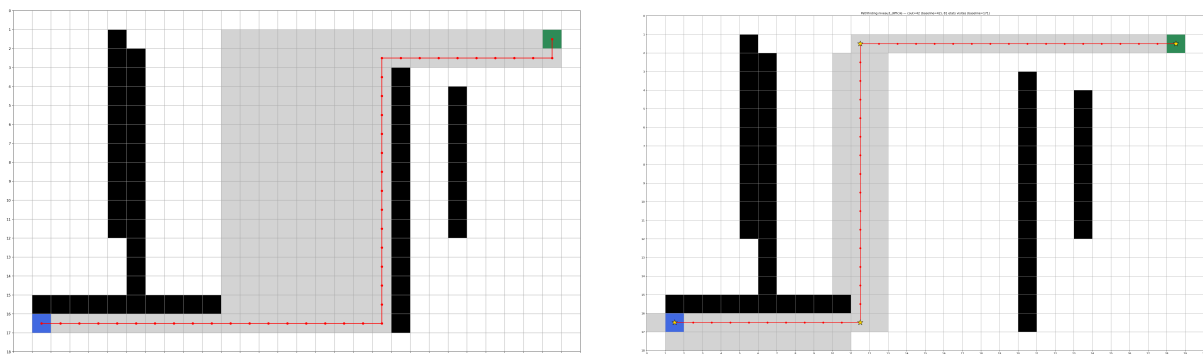


FIGURE 4 – Recherche de chemin niveau3_difficile. À gauche, A* (171 états visités). À droite, LLM-A* avec Claude (81 états visités), à coût identique, avec 2 points de passage intermédiaires repérant le couloir libre du labyrinthe.

Les points 1 et 2 gagnent peu, quelques états en moins sur niveau1 et niveau2, un effet nul sur niveau3 une fois le budget de 150 appels épuisé. LLM-A* gagne beaucoup plus, jusqu'à diviser par 2 à 3 le nombre d'états visités selon l'instance, pour un seul appel API au lieu de 150.

4.4 Résultats pour Taquin

Instance	Variante	Coût	Visites	Appels API
niveau2_moyen	A*	13	55	0
niveau2_moyen	point 1 (departager_llm)	13	54	49
niveau2_moyen	point 2 (heuristique_llm_lot)	13	55	54
niveau2_moyen	LLM-A*	13	30 (Claude), 38 (DeepSeek)	1
niveau3_difficile	A*	16	48	0
niveau3_difficile	point 1 (departager_llm)	16	48	36
niveau3_difficile	point 2 (heuristique_llm_lot)	16	48	49
niveau3_difficile	LLM-A*	20 (Claude), 16 (DeepSeek)	241 (Claude), 170 (DeepSeek)	1

TABLE 4 – Taquin, tous les mécanismes comparés (mêmes variantes que pour Recherche de chemin). Points 1 et 2 avec DeepSeek. LLM-A* avec Claude et DeepSeek. Sur niveau3, Claude casse l'optimalité, coût 20 au lieu de l'optimum réel 16, et les deux fournisseurs visitent bien plus d'états que le baseline.

Les points 1 et 2 restent optimaux sur les deux niveaux, comme attendu théoriquement, mais l'effet mesuré reste très faible. Sur niveau2_moyen, LLM-A* reste optimal et visite moins d'états, avec de vrais points de passage. Sur niveau3_difficile, le tableau se retourne. Claude casse l'optimalité, reproductible sur les 3 cartes testées (table 4 et essais complémentaires). Les deux fournisseurs visitent en plus bien plus d'états que le baseline, jusqu'à 5 fois plus, alors même qu'ils proposent de vraies dispositions intermédiaires. Le raisonnement du LLM n'est donc pas seulement insuffisant sur ce niveau, il devient contre-productif.

4.5 Résultats pour Sokoban

Instance	Variante	Coût	Visites	Appels API
microban niveau0	A*	1	1	0
microban niveau0	point 1 (departager_llm)	1	1	0
microban niveau0	point 2 (heuristique_llm_lot)	1	1	2
microban niveau0	point 3 (elaguer_llm)	1	1	1
microban niveau0	LLM-A*	1	1 (Claude, DeepSeek)	1
microban niveau1	A*	30	1088	0
microban niveau1	point 1 (departager_llm)	30	1088	150
microban niveau1	point 2 (heuristique_llm_lot)	30	1088	150
microban niveau1	point 3 (elaguer_llm)	30	1088	150
microban niveau1	LLM-A*	30	819 (Claude, DeepSeek)	1
microban niveau2	A*	66	1761	0
microban niveau2	point 1 (departager_llm)	66	1761	150
microban niveau2	point 2 (heuristique_llm_lot)	66	1761	150
microban niveau2	point 3 (elaguer_llm)	66	1761	150
microban niveau2	LLM-A*	66	1814 (Claude), 1626 (DeepSeek)	1
microban niveau3	A*	81	13 428	0
microban niveau3	point 1 (departager_llm)	81	13 428	150
microban niveau3	point 2 (heuristique_llm_lot)	81	13 428	150
microban niveau3	point 3 (elaguer_llm)	81	13 428	150
microban niveau3	LLM-A*	81	11 671 (Claude), 13 540 (DeepSeek)	1
microban niveau4	A*	89	56 240	0
microban niveau4	point 1 (departager_llm)	89	56 240	150
microban niveau4	point 2 (heuristique_llm_lot)	89	56 240	150
microban niveau4	point 3 (elaguer_llm)	89	56 240	150
microban niveau4	LLM-A*	89	51 065 (Claude), 50 924 (DeepSeek)	1
original niveau1	A*	échec	250 000 (plafond)	0
original niveau1	point 1 (departager_llm)	<i>non mesurable</i>	–	–
original niveau1	point 2 (heuristique_llm_lot)	<i>non tenté</i>	–	–
original niveau1	point 3 (elaguer_llm)	<i>non tenté</i>	–	–
original niveau1	LLM-A*	<i>non mesurable</i>	250 000 (plafond)	1

TABLE 5 – Sokoban, tous les mécanismes comparés. Points 1, 2 et 3 avec DeepSeek. LLM-A* avec Claude et DeepSeek. Sur niveau2 et niveau3, un des deux fournisseurs visite plus d'états que le baseline, en gras.

Les points 1, 2 et 3 n'ont ici strictement aucun effet mesurable, sur aucun niveau microban. Le budget de 150 appels API est négligeable face à des espaces de 1 000 à 56 000 états et plus, si bien que le LLM n'influence qu'une poignée de décisions précoces, jamais suffisant pour changer une seule décision mesurable. LLM-A*, à l'inverse, ne casse jamais l'optimalité ici, même avec de vraies dispositions intermédiaires. Le nombre d'états visités baisse sur la plupart des niveaux, mais pas partout. Sur niveau2 avec Claude et niveau3 avec DeepSeek, LLM-A* visite plus d'états que le baseline, malgré un vrai raisonnement à chaque fois. Sur original niveau1, le seul appel de LLM-A* ne suffit de toute façon pas à éviter le mur des 250 000 états, comme la baseline.

4.6 Réponse à la problématique

Revenons à la question de recherche posée en introduction. Peut-on insérer un LLM à des points précis d'un algorithme A* sans casser l'optimalité, et à quel prix réel, en appels API et en gain mesuré ?

Pour les points 1 et 2, l’optimalité reste garantie. Le prix payé est élevé, jusqu’à 150 appels API par instance, pour un gain mesuré nul. Ils ne fonctionnent tout simplement pas, une fois bornés par un budget d’appels réaliste. Les garanties théoriques posées dessus semblent finalement plus brider le LLM qu’autre chose.

LLM-A*, à l’inverse, exploite la pleine puissance du LLM, précisément parce qu’il ne le bride pas. Il ne coûte qu’un seul appel API par instance. Sur Recherche de chemin, il réduit presque toujours le nombre d’états visités, à coût optimal. Sur Sokoban, le coût reste toujours optimal, mais le nombre d’états visités ne baisse pas systématiquement. Sur Taquin niveau3_difficile, le risque se matérialise vraiment. L’optimalité casse parfois, et le nombre d’états visités augmente le plus souvent, jusqu’à 5 fois plus qu’un A* classique, même avec un vrai raisonnement du LLM.

Le jeu compte aussi beaucoup, tout comme la façon dont on décrit son état. Le nombre de combinaisons possibles joue également. Sur Recherche de chemin, l’espace de coordonnées reste simple, et LLM-A* y est presque toujours utile. Sur Taquin et Sokoban, l’espace de dispositions complètes est bien plus grand, et le gain devient inconstant, parfois même négatif.

Généraliser ce mécanisme à tous les problèmes reste donc compliqué. Sokoban original niveau1 se heurte au même mur de 250 000 états que la baseline classique, même avec LLM-A*, précisément le jeu le plus difficile de nos trois. Taquin niveau3_difficile montre en plus qu’un vrai raisonnement du LLM ne suffit pas toujours à aider la recherche, et peut même la ralentir.

Références

- [1] Meng, S., Wang, Y., et al. (2024). *LLM-A* : Large Language Model Enhanced Incremental Heuristic Search on Path Planning*. [arXiv:2407.02511](#).